

برنامه‌سازی پیشرفته

تمرین کامپیوتری شماره ۲



مدرس: رامتین خسروی

طراحان: پارسا سعیدنیا، کوثر شیرینی جعفرزاده، فرجاد
فلاح، مهدی نائینی، پارسا دقیق، بهاره عین اللهی، فرشته
باقری

مهلت تحویل: دوشنبه ۵ آبان ۱۴۰۴، ساعت ۲۳:۵۹

مقدمه

هدف از این تمرین آشنایی شما با طراحی برنامه است. با توجه به حجم تمرین توصیه می‌شود این روال را برای انجام آن طی کنید.

1. قبل از شروع پیاده‌سازی زمانی را به خواندن کامل شرح مسئله اختصاص دهید.
 2. پس از آن به نحوه شکست برنامه به اجزای کوچکتر (مثل روال‌های خواندن ورودی اولیه و رسیدگی به دستورات کاربر) فکر کنید.
 3. بر اساس شکست بند قبل، درباره تایپ‌هایی که اطلاعات ثابت برنامه (در اینجا اطلاعات آهنگ‌ها و کاربران) را نگه می‌دارند تصمیم‌گیری کنید.
 4. توصیه می‌شود به این ترتیب پیاده‌سازی خود را پیش ببرید:
 - a. ابتدا خواندن فایل‌های ورودی را کامل کنید.
 - b. دستورها را یکی یکی در برنامه خود پیاده‌سازی کنید.
 - c. بعد از پیاده‌سازی هر دستور درستی کارکرد برنامه خود را با داده‌های آزمون بیازمایید.
- به این ترتیب، حتی اگر فرصت نکردید تمام برنامه را پیاده‌سازی کنید آنچه تا پایان مهلت تمرین دارید برنامه‌ای است که کار می‌کند (هر چند کامل نیست).
- برای آشنایی بیشتر با مفاهیم طراحی بالا به پایین می‌توانید به ویدیویی که در بخش محتوای دستیاران آموزشی در [صفحه درس](#) یا [وبسایت](#) درس مراجعه کنید.

اسیوتیفای

در این برنامه، شما وظیفه دارید یک سیستم پیشنهاد موسیقی را پیاده‌سازی کنید. این سیستم به ازای هر کاربر تاریخچه آهنگ‌هایی که گوش داده است را ذخیره می‌کند و بر اساس ویژگی‌های مختلف آهنگ‌ها، آهنگ مناسب را به او پیشنهاد می‌دهد و یا یک پلی‌لیست از آهنگ‌ها برای او ایجاد می‌کند. همچنین امکان گوش دادن به آهنگ‌ها و پلی‌لیست‌های خودش یا کاربران دیگر را فراهم می‌کند.

نحوه اجرای برنامه

در زمان اجرای برنامه مسیر فایل‌های ورودی به ترتیب (ابتدا مسیر فایل آهنگ‌ها، سپس مسیر فایل کاربران) از طریق آرگومان خط فرمان¹ به برنامه داده می‌شود. برای آشنایی با این نوع آرگومان‌ها، می‌توانید [این لینک](#) را مشاهده کنید.

قالب ورودی
<code>./Sotify.out </path/to/songs/file> </path/to/users/file></code>

قالب فایل‌های ورودی

اطلاعات مربوط به آهنگ‌ها و کاربران در دو فایل جداگانه قرار دارند. آدرس این فایل‌ها به صورت آرگومان خط فرمان به برنامه شما داده می‌شوند. شما باید اطلاعات موجود در آن‌ها را در ابتدای اجرا برنامه خوانده و ذخیره کنید. نوع فایل‌های ورودی به صورت CSV² هستند و برای آشنایی با این نوع فایل‌ها می‌توانید [این لینک](#) را بررسی کنید.

در ادامه شرح هر یک از فایل‌ها آمده است. هر فایل CSV یک سطر header در ابتدای خود دارد که نام هر یک از ستون‌ها را به ترتیب مشخص می‌کند.

¹ Command line

² Comma-Separated Values

فایل آهنگ‌ها

در این فایل پنج ستون نام، شناسه، سال انتشار، سبک و سازنده آهنگ وجود دارد. تضمین می‌شود که شناسه هر آهنگ یکتاست و در نام، سبک و سازنده آهنگ کاراکتر فاصله³ وجود ندارد.

نام ویژگی	توضیحات	نوع داده
Name	نام آهنگ	رشته
ID	شناسه آهنگ	عدد طبیعی
Year	سال انتشار آهنگ	عدد طبیعی
Genre	سبک آهنگ	رشته
Artist	سازنده آهنگ	رشته

نمونه فایل آهنگ‌ها
Name, ID, Year, Genre, Artist Fragile_Dreams, 1, 1999, Progressive_Rock, Anathema A_Simple_Mistake, 2, 2010, Progressive_Rock, Anathema Head_Above_Water, 3, 2018, Pop_Rock, Avril_Lavigne Time, 4, 2010, Film_Score, Hans_Zimmer Cornfield_Chase, 5, 2014, Film_Score, Hans_Zimmer Symphony_No._5, 6, 1808, Classical, Ludwig_van_Beethoven

³ Space

فایل کاربران

در این فایل سه ستون نام کاربری، لیستی از شناسه آهنگ‌ها به همراه تعداد دفعاتی که کاربر به آن آهنگ گوش داده و سبک مورد علاقه کاربر وجود دارد. تضمین می‌شود که در نام کاربری و سبک مورد علاقه کاربر، کاراکتر فاصله وجود ندارد. در لیست آهنگ‌ها، کاراکتر «;»⁴ نقش جداکننده بین تاریخچه آهنگ‌ها را دارد. تاریخچه هر آهنگ شامل شناسه آهنگ و تعداد دفعاتی است که کاربر به آن گوش داده است. کاراکتر «:»⁵ نقش جداکننده بین شناسه آهنگ و تعداد دفعاتی که کاربر قبلاً به آن گوش داده را دارد. تضمین می‌شود که از کاراکتر «:» و «;» در جای دیگری استفاده نمی‌شود.

نوع داده	توضیحات	نام ویژگی
رشته	نام کاربری	Username
رشته	تاریخچه آهنگ‌های کاربر	Music_History
رشته	سبک مورد علاقه کاربر	Favorite_Genre

نمونه فایل کاربران
Username,Music_History,Favorite_Genre Kosar,6:8;2:1,Classical Parsa,4:12;1:2,Film_Score Koroush,1:15;2:10;4:3,Progressive_Rock Parisa,3:20;1:5;6:2,Pop_Rock Farjad,4:6;2:4;3:3;6:1,Film_Score

⁴ Semicolon

⁵ Colon

دستورات برنامه

دستور گوش دادن به آهنگ

با وارد کردن این دستور، کاربری که نام کاربری آن در ورودی داده می‌شود، یکبار به آهنگی که شناسه آن وارد شده گوش می‌دهد و لذا تعداد دفعاتی که کاربر به آن آهنگ گوش داده یک واحد افزایش می‌یابد. تضمین می‌شود نام کاربری و شناسه آهنگ وارد شده، موجود می‌باشد.

قالب ورودی
listen_song <Username> <Song_ID>
قالب خروجی
<Username> listened to <Song_Name>!

نمونه ورودی
listen_song Farjad 5
نمونه خروجی
Farjad listened to Cornfield_Chase!

دستور ایجاد پلی‌لیست

این دستور با گرفتن نام پلی‌لیست، نام کاربر ایجاد کننده آن و تعداد آهنگ‌های موجود در آن، پلی‌لیست مورد نظر را ساخته و آهنگ‌های موجود در آن را بر اساس امتیاز آنها نمایش می‌دهد. برای ایجاد پلی‌لیست، تنها از آهنگ‌هایی که کاربر به آن‌ها گوش نداده است استفاده می‌کنیم و نباید آهنگی که کاربر قبلاً حداقل یک بار به آن گوش داده در پلی‌لیست قرار بگیرد.

برای انتخاب میان این آهنگ‌ها، امتیازات زیر را محاسبه می‌کنیم.

- امتیاز سازنده آهنگ برابر مجموع دفعاتی است که کاربر به آهنگ‌هایی از آن سازنده گوش کرده است.
- امتیاز سال برابر مجموع دفعاتی است که کاربر به آهنگ‌های منتشر شده در آن سال گوش داده است.
- امتیاز سبک برابر مجموع دفعاتی است که کاربر به آهنگ‌هایی با آن سبک گوش داده است.

علاوه بر این سه نوع امتیاز برای سبک مورد علاقه کاربر ایجاد کننده، ۵ امتیاز اضافه در نظر می‌گیریم. به این شکل که در محاسبه امتیاز نهایی آهنگ جدید، اگر سبک آن همان سبک مورد علاقه کاربر باشد، ۵ امتیاز بیشتر علاوه بر امتیازات سازنده و سال و سبک که گفته شد، به آن تعلق می‌گیرد.

امتیاز نهایی هر آهنگ برابر حاصل جمع امتیازات گرفته شده در هر قسمت می‌باشد. در صورتی که امتیاز دو آهنگ برابر باشد، بر اساس نام آهنگ انتخاب و مرتب می‌کنیم. در انتها نحوه مرتب سازی بر اساس نام توضیح داده شده است. تضمین می‌شود به تعداد خواسته شده در دستور، آهنگ جدید وجود دارد.

قالب ورودی
<code>create_playlist <Playlist_Name> <Creator_Username> <Songs_Count></code>
قالب خروجی
<code><Playlist_Name>: <Song_Name1> <Artist1> <Year1> <Genre1> <Song_Name2> <Artist2> <Year2> <Genre2> ...</code>

نمونه ورودی
<code>create_playlist Dream_Within_A_Dream Parsa 3</code>
نمونه خروجی
<code>Dream_Within_A_Dream: Cornfield_Chase Hans_Zimmer 2014 Film_Score A_Simple_Mistake Anathema 2010 Progressive_Rock Head_Above_Water Avril_Lavigne 2018 Pop_Rock</code>

توضیح مثال

کاربر Parsa به آهنگ های Time و Fragile_Dreams به ترتیب ۱۲ و ۲ بار گوش کرده است. پس آهنگ های دیگر را بررسی می کنیم:

مجموع	امتیاز سبک مورد علاقه	امتیاز سال	امتیاز سازنده	امتیاز سبک	نام آهنگ
16	0	12	2	2	A_Simple_Mistake
0	0	0	0	0	Head_Above_Water
29	5	0	12	12	Cornfield_Chase
0	0	0	0	0	Symphony_No._5

در محاسبه امتیازات آهنگ A_Simple_Mistake، کاربر Parsa دوبار به سبک Progressive_Rock گوش کرده پس این آهنگ ۲ امتیاز سبک می گیرد. همچنین ۲ بار به آهنگ های Anathema گوش کرده پس ۲ امتیاز سازنده نیز می گیرد. برای امتیاز سال چون ۱۲ بار به آهنگ Time گوش کرده که سال تولید برابری با این آهنگ دارد، ۱۲ امتیاز سال نیز دریافت می کند. در نهایت چون سبک مورد علاقه او Progressive_Rock نیست، امتیاز سبک مورد علاقه تعلق نمی گیرد.

پس از محاسبه مجموع امتیازات هر آهنگ، واضح است که آهنگ Cornfield_Chase و A_Simple_Mistake بیشترین امتیاز را گرفته اند و در لیست آهنگ های پلی لیست به ترتیب جایگاه اول و دوم را می گیرند ولی برای آهنگ سوم پلی لیست باید از میان دو آهنگ دیگر انتخاب کنیم که هر دو امتیاز برابر صفر دارند. پس بر اساس نام آهنگ باید انتخاب کنیم که در این صورت آهنگ Head_Above_Water برگزیده می شود.

دستور گوش دادن به پلی لیست

با وارد کردن این دستور کاربری که در ورودی داده می شود، به تمامی آهنگ های پلی لیست مورد نظر خود گوش می دهد. لازم به ذکر است که یک کاربر می تواند به پلی لیست دیگر کاربران نیز گوش دهد. دو کاربر می توانند پلی لیستی با نام یکسان داشته باشند. بنابراین برای گوش دادن به یک پلی لیست، نیاز است نام پلی لیست و نام کاربر ایجاد کننده آن وارد شود. تضمین می شود نام کاربری، نام پلی لیست و نام کاربر ایجاد کننده وارد شده، موجود می باشد و کاربر ایجاد کننده پلی لیستی با آن نام دارد.

قالب ورودی
listen_playlist <Username> <Playlist_Name> <Creator_Username>
قالب خروجی
<Username> listened to the <Playlist_Name> playlist!

نمونه ورودی
listen_playlist Parsa Dream_Within_A_Dream Parsa
نمونه خروجی
Parsa listened to the Dream_Within_A_Dream playlist!

دستور پیشنهاد آهنگ

این دستور با گرفتن نام کاربری، یک آهنگ به او پیشنهاد می‌دهد. برای این دستور نیز از همان روش امتیازدهی که در دستور ایجاد پلی‌لیست گفته شد، استفاده می‌کنیم و از بین آهنگ هایی که کاربر به آنها گوش نکرده، آهنگی را که بیشترین امتیاز را داشته انتخاب کرده و اطلاعات آن را نمایش می‌دهیم. تضمین می‌شود نام کاربری وارد شده وجود دارد.

قالب ورودی
recommend_song <Username>
قالب خروجی
Recommended song for <Username>: <Song_Name> <Artist> <Year> <Genre>

نمونه ورودی
recommend_song Koroush
نمونه خروجی
Recommended song for Koroush: Cornfield_Chase Hans_Zimmer 2014 Film_Score

نحوه مرتب‌سازی بر اساس نام

در زبان C++، هر رشته شامل تعدادی کاراکتر است و هر کاراکتر دارای یک کد ASCII است. این کد برای حروف انگلیسی کوچک از ۹۷ تا ۱۲۲ و برای حروف انگلیسی بزرگ از ۶۵ تا ۹۰ است. جدول کامل ASCII را می‌توانید در [این لینک](#) مشاهده کنید.

با استفاده از این روش کدگذاری، می‌توانیم یک روش مقایسه رشته‌ها را به صورت مقایسه این کدها با یکدیگر در نظر بگیریم. این روش به این صورت است که کد ASCII کاراکترهای اول دو رشته با همدیگر مقایسه می‌شوند، در صورتی که یکی از آن‌ها کوچک‌تر از دیگری باشد، کل آن رشته کوچک‌تر از رشته دیگر در نظر گرفته می‌شود. در صورتی که کاراکترهای اول دو رشته با همدیگر برابر باشند، کاراکترهای دوم را مقایسه می‌کنیم. این کار را تا زمانی انجام می‌دهیم که به کاراکتری برسیم که در دو رشته یکسان نباشد. در صورتی که پیش از رسیدن به چنین کاراکتری به انتهای هر دو رشته برسیم، دو رشته با همدیگر برابر در نظر گرفته می‌شوند. در زبان C++، [اپراتورهای مقایسه‌ای](#) نیز برای رشته‌ها دقیقاً به همین صورت پیاده‌سازی شده‌اند.

در این تمرین، در بخش‌هایی که خواسته شده آیت‌ها را بر اساس نام مرتب کنید، لازم است دقیقاً به همین صورت عمل کنید. برای مثال نام‌های زیر را در نظر بگیرید:

نمونه نام‌ها
Amir, amin, Borna, dorsa, AMirreza

در صورت مرتب‌سازی، نام‌ها به صورت زیر خواهند بود:

نمونه نام‌های مرتب‌شده
AMirreza, Amir, Borna, amin, dorsa

نکات و نحوه تحویل

- تحویل این تمرین در سامانه گیت‌هاب انجام می‌شود. برای انجام تمرین لطفاً از طریق [این لینک](#) وارد شوید. پس از آن باید شماره دانشجویی خود را انتخاب کنید (دقت کنید که با کمک این شماره دانشجویی به شما نمره خواهیم داد، لطفاً در انتخاب درست شماره دانشجویی حتماً دقت کنید، در صورتی که به مشکل خوردید با دستیاران در ارتباط باشید). پس از آن به صفحه‌ای منتقل می‌شوید که در آنجا می‌توانید تمرین جدید را قبول کنید. بعد از قبول کردن تمرین یک مخزن⁶ در [AP Assignments](#) برای شما ساخته می‌شود و باید کدهای خود را در آنجا قرار دهید. لطفاً دقت کنید که نهایی خود را حتماً در فایل `src/main.cpp` قرار دهید.

- پس از انجام تمرین و بارگذاری در گیت‌هاب، کد Hash آخرین کامیت⁷ را به همراه شماره دانشجویی خود در سامانه ایلرن آپلود کنید (در خط اول شماره دانشجویی، پس از آن از `Enter` استفاده کنید و به خط بعد بروید و پس از آن Hash آخرین کامیت). نمونه متن خواسته شده در سامانه ایلرن (بخش `<last_commit_hash>` و `<sid>` را جایگزین کنید):

```
<sid>
<last_commit_hash>
```

نمونه:

```
810100000
bad8fbcdfcf3b9feb371a31e0c370150aa870b18
```

- دقت کنید که عدم رعایت ساختار گفته شده در آپلود یا تغییر ساختار فایل‌ها در مخزن (می‌توانید به دلخواه خود فایل اضافه کنید و ... اما اسم و ساختار فایل‌هایی که در ابتدا به شما داده می‌شود نباید تغییر کند) باعث کسر ۱۰ درصد از نمره شما خواهد شد.
- سعی کنید مرحله به مرحله کد را کامیت کنید و از کامیت کردن کل کد در انتهای پروژه خودداری کنید. همچنین، پیام کامیت‌های شما باید خوانا باشد و تغییر ایجاد شده را توضیح دهد. دقت کنید که بخشی از نمره شما را استفاده صحیح از گیت تشکیل می‌دهد. برای مطالعه در خصوص نحوه صحیح کامیت کردن، می‌توانید [این لینک](#) را مطالعه کنید.
- برنامه شما باید در سیستم عامل لینوکس و با مترجم `g++` با استاندارد `c++20` ترجمه و در زمان معقول برای ورودی‌های آزمون اجرا شود.

⁶ Repository

⁷ Commit

- بخش مهمی از ارزیابی برنامه شما درستی عملکرد آن است. بنابراین به اندازه کافی ورودی‌های آزمایشی طراحی کنید تا درستی خروجی در حالت‌های مختلف آزموده شود و دقت کنید که اگر برنامه شما با مثال‌های داده شده درست کار کند لزوماً به معنای درستی در تمام سناریوها نیست.
- درستی برنامه شما از طریق آزمون‌های خودکار سنجیده می‌شود. به این ترتیب، لازم است خروجی تولید شده از نظر بزرگی و کوچکی حروف، رعایت فاصله‌ها، عدم وجود خروجی‌های اضافه، ... دقیقاً مانند نمونه‌های داده شده باشد. بنابراین پیشنهاد می‌شود که با استفاده از ابزارهایی مانند diff فرمت خروجی برنامه خود را با خروجی‌هایی که در اختیارتان قرار داده شده است مطابقت دهید.
- هدف این تمرین یادگیری شماست. لطفاً تمرین را خودتان انجام دهید. در صورت کشف تقلب مطابق سیاست درس با آن برخورد خواهد شد.

نمرات

- تمیزی کد

- رعایت کردن نام‌گذاری صحیح و منسجم بودن⁸
- عدم وجود کد تکراری
- رعایت دندانه‌گذاری⁹
- استفاده صحیح از متغیرهای ثابت¹⁰ به جای Magic Value-ها¹¹
- استفاده مناسب از container-ها و iterator-ها به جای روش‌های قدیمی
- استفاده مناسب از ساختار داده‌های مناسب
- توابع کوتاه که فقط یک کار را انجام می‌دهند

- درستی کد

- آزمون‌های خودکار

- طراحی

- شکستن مناسب و مرحله به مرحله مسئله
- ذخیره اطلاعات در ساختار داده‌های مناسب
- ساختاردهی کد در قالب توابع کوتاه که فقط یک کار را انجام می‌دهند
- پرهیز از متغیرهای سراسری

- گیت

- پیام‌های خوانا برای commit-ها
- Commit کردن مرحله به مرحله

دقت کنید که موارد ذکر شده لزوماً کل نمره شما را تشکیل نمی‌دهند و ممکن است با تغییراتی همراه باشند.

⁸ Consistency

⁹ Indentation

¹⁰ Constant

¹¹ به مقادیر خاصی که در کد استفاده می‌شود و برای عملکرد صحیح کد ضروری است اما دلیل استفاده از آن‌ها مشخص نیست و قابل جایگزین شدن با یک ثابت یا اسم مشخص جهت افزایش خوانایی هستند، magic value گفته می‌شود. برای آشنایی بیشتر با این مفهوم می‌توانید [این لینک](#) را مشاهده کنید.